ÉCOLE NATIONALE SUPÉRIEURE D'ARTS ET MÉTIERS
UNIVERSITÉ HASSAN II DE CASABLANCA

Module : Python avancé

Rapport de Projet

MediGest

Application de gestion d'une clinique

Python · SQLite · Tkinter

Programmation orientée objet • base de données
Cas d'étude : Clinique Al Amal (noms et chiffres fictifs)

Préparé par :

- **ELGOTAI** Abdessamad
- **ELBERHICHI** Abderrahmane
- **AKRINE** Youssef

Sous la direction de :
Pr. Mustapha HAIN

Filière et section :
IAGI | 1ère Année Cycle

Année académique :
2026 — 2027

Remerciements

Au terme de ce projet, nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à sa réalisation.

Nous remercions tout particulièrement **Monsieur le Professeur Mustapha HAIN**, notre enseignant du module « Python avancé », pour la clarté de ses cours, la qualité de son encadrement et la liberté qu'il nous a laissée pour proposer un projet qui nous ressemble. Ses consignes claires — trois tables, une interface Tkinter, de la programmation orientée objet — nous ont poussés à chercher l'essentiel plutôt que le superflu.

Nos remerciements vont aussi à l'équipe pédagogique et administrative de la formation, à nos camarades pour leurs retours lors des présentations intermédiaires, à nos familles pour leur patience et leur soutien, ainsi qu'aux communautés qui documentent librement les outils utilisés : Python, SQLite et Tkinter.

Nous espérons sincèrement que ce travail sera à la hauteur de ses exigences et de la confiance qu'il nous a accordée. Qu'il trouve ici l'expression de notre respect et de notre sincère reconnaissance.

Les membres du groupe

Résumé

MediGest est une application de bureau, écrite en Python, qui remplace le cahier papier d'une clinique (la Clinique Al Amal, cas d'étude fictif) par un outil unique et fiable. Le **médecin** y gère ses patients, ses rendez-vous, ses consultations et ordonnances, le stock de médicaments, la facturation et un tableau de bord. Le **patient** consulte uniquement ses propres rendez-vous, ordonnances et factures, et peut demander un rendez-vous.

Le projet repose sur trois piliers :

- une **base de données SQLite** de seulement **3 tables** (`Users`, `rendez_vous`, `medicament`) qui refuse elle-même les données incohérentes ;
- une **conception orientée objet** fondée sur l'**héritage** : une classe mère `Personne` dont héritent `Patient` et `Medecin` ;
- une **interface Tkinter** de 10 écrans (7 onglets pour le médecin, 4 pour le patient) avec des graphiques dessinés à la main sur `Canvas`.

Sur le jeu d'essai, le tableau de bord montre par exemple **17 visites** dont 12 honorées, **2 319 MAD** encaissés contre **212 MAD** d'impayés, et **2 médicaments sous leur seuil d'alerte**. Aucune bibliothèque externe n'est nécessaire.

Mots-clés : gestion de clinique, Python, SQLite, Tkinter, héritage, POO, DAO, tableau de bord.

Table des matières

Remerciements ... **2**
Résumé ... **2**
I. Introduction et contexte ... **4**
1.1 Le problème à résoudre ... 4
1.2 La solution : MediGest ... 4
1.3 Cas d'étude : la Clinique Al Amal ... 4
II. Objectifs du projet ... **4**
III. Cahier des charges ... **5**
3.1 Les deux profils d'utilisateurs ... 5
3.2 Besoins fonctionnels et non fonctionnels ... 5
3.3 Fonctionnalités principales ... 5
3.4 Parcours de l'utilisateur ... 6
IV. Outils et démarche de réalisation ... **6**
4.1 Technologies utilisées ... 6
4.2 Démarche de réalisation ... 6
V. Architecture et description des fichiers ... **7**
5.1 Une architecture en 3 couches ... 7
5.2 Rôle de chaque fichier ... 7
VI. Le principe d'héritage ... **8**
6.1 Les classes du modèle ... 8
6.2 Le code correspondant (extrait de `models.py`) ... 8
6.3 Ce que l'on obtient grâce à l'héritage ... 9
6.4 Pourquoi c'est utile ... 9
6.5 Les autres notions de POO utilisées ... 9
VII. Conception de la base de données ... **10**
7.1 Démarche en 4 étapes ... 10
7.2 Schéma de la base ... 11
7.3 Description des 3 tables ... 11
7.4 Les protections intégrées à la base ... 13
7.5 Le cycle de vie d'un rendez-vous ... 13
7.6 Interprétation des choix de conception ... 13
VIII. Les classes d'accès aux données (DAO) ... **14**
IX. Interfaces graphiques ... **14**
9.1 Composants de l'interface ... 15
9.2 Connexion et espace médecin ... 15
9.3 Espace patient ... 16
9.4 Le tableau de bord ... 17
X. Sécurité et protection des données ... **17**
10.1 Le hachage des mots de passe ... 17
10.2 Risques et protections ... 18
XI. Jeu d'essai et scénarios de validation ... **18**
11.1 Le jeu d'essai ... 18
11.2 Scénarios de validation ... 18
XII. Résultats et interprétation ... **19**
12.1 Lecture des indicateurs ... 20
12.2 Chiffre d'affaires et stock ... 20
12.3 Difficultés rencontrées et solutions ... 20
12.4 Limites et perspectives ... 21
Annexe : lancer l'application ... **21**
Conclusion générale ... **21**

Rapport de projet • Module Python avancé

MediGest : application de gestion d'une clinique

Partie A — Présentation et conception

I. Introduction et contexte

1.1 Le problème à résoudre

Dans beaucoup de petites cliniques, les activités sont encore notées sur un cahier papier. Les conséquences sont bien connues : deux patients notés à la même heure, un médicament prescrit alors qu'il n'est plus en stock, une consultation jamais facturée, un dossier introuvable le jour où l'on en a besoin. Ces erreurs coûtent du temps, de l'argent et peuvent même affecter la qualité des soins.

1.2 La solution : MediGest

MediGest est une application de bureau qui remplace le cahier par un seul outil. Elle ne se contente pas d'afficher des écrans : les règles importantes sont vérifiées à la fois par le code Python et par la base de données. Un identifiant, un CIN ou un e-mail ne peut pas être utilisé deux fois, un stock ne peut jamais devenir négatif, un prix ne peut pas être négatif, et un patient ne peut pas être son propre médecin.

> MediGest, c'est **le cahier de la clinique, mais numérique et intelligent** : il range les personnes, les rendez-vous et les médicaments dans trois « classeurs », il empêche les erreurs courantes, et il affiche en un coup d'œil l'état de la clinique.

1.3 Cas d'étude : la Clinique Al Amal

Pour donner un cadre concret au projet, nous avons imaginé la **Clinique Al Amal** à Casablanca. Elle emploie deux médecins (le Dr Karim Alaoui en médecine générale, tarif de consultation 200 MAD, et la Dr Salma Bennani en cardiologie, tarif 300 MAD) et suit des patients dont les noms et les chiffres sont entièrement fictifs.

II. Objectifs du projet

Objectif général. Concevoir une application complète, simple à utiliser, qui informatise la gestion quotidienne d'une clinique, tout en mettant en pratique les notions du module « Python avancé ».

Objectifs spécifiques :

- **Organiser** les patients, les rendez-vous et les dossiers dans une base de données fiable ;
- **Sécuriser** l'accès : mots de passe hachés, chacun ne voit que ce qui le concerne ;
- **Faciliter** le travail du médecin : planification, consultation, ordonnance, facture, stock ;
- **Informer** : alertes de stock bas, impayés, tableau de bord avec graphiques ;
- **Appliquer** les notions du cours : classes, héritage, exceptions, SQL, interface graphique ;
- **Respecter** les consignes : 3 tables au maximum, programmation orientée objet, interface Tkinter.

III. Cahier des charges

3.1 Les deux profils d'utilisateurs

Profil	Ce qu'il peut faire	Ce qu'il voit
Médecin (7 onglets)	Crée et archive les comptes patients, planifie ou confirme les rendez-vous, saisit consultations et ordonnances, facture et encaisse, gère le stock, consulte le tableau de bord, sauvegarde la base.	Ses rendez-vous, la liste des patients, le stock, les statistiques.
Patient (4 onglets)	Consulte ses rendez-vous, ordonnances et factures, demande un rendez-vous parmi les créneaux libres, modifie ses coordonnées et son mot de passe.	Uniquement ses propres données.

Tab. *1. Les deux profils de l'application.*

3.2 Besoins fonctionnels et non fonctionnels

Type	Besoin
Fonctionnel	Se connecter avec un identifiant et un mot de passe ; gérer les comptes patients (création, recherche, archivage) ; prendre, planifier et annuler des rendez-vous ; enregistrer une consultation avec diagnostic et ordonnance ; facturer et encaisser ; suivre le stock et les alertes ; imprimer ordonnance et facture, exporter en CSV ; afficher des statistiques ; sauvegarder la base.
Données	Garder un historique fiable ; ne jamais supprimer un patient ayant des rendez-vous ; refuser les doublons (CIN, login, e-mail) et les valeurs impossibles (stock ou prix négatifs).
Sécurité	Aucun mot de passe en clair ; droits d'écriture réservés au médecin ; chaque patient ne voit que son dossier.
Ergonomie	Interface en français, couleurs porteuses de sens, messages d'erreur clairs sans code technique, fenêtre redimensionnable.
Technique	Python 3 et bibliothèque standard uniquement ; base locale SQLite ; code organisé en couches et classes.

Tab. *2. Cahier des charges de MediGest.*

3.3 Fonctionnalités principales

- **Connexion sécurisée** : l'application ouvre les bons écrans selon le rôle du compte ; un compte archivé ne peut plus se connecter.
- **Rendez-vous** : créneaux de 30 minutes entre 08 :00 et 17 :30, seuls les créneaux libres sont proposés ; filtres « aujourd'hui / cette semaine / tous » ; un patient a au plus 2 demandes en attente et ne peut annuler qu'au moins 24 h avant l'heure prévue.
- **Consultation et ordonnance** : diagnostic obligatoire, lignes de médicaments avec posologie, rappel des allergies du patient, vérification du stock ; la validation met le stock à jour.
- **Facturation** : tarif du médecin plus prix des médicaments, encaissement (espèces, carte, chèque), impression HTML et export CSV.
- **Stock** : alerte lorsque la quantité est inférieure ou égale au seuil, réapprovisionnement, ajout de médicaments.

— **Tableau de bord** : indicateurs clés, histogramme, anneau des statuts, médicaments les plus prescrits, sauvegarde de la base en un clic.

3.4 Parcours de l'utilisateur

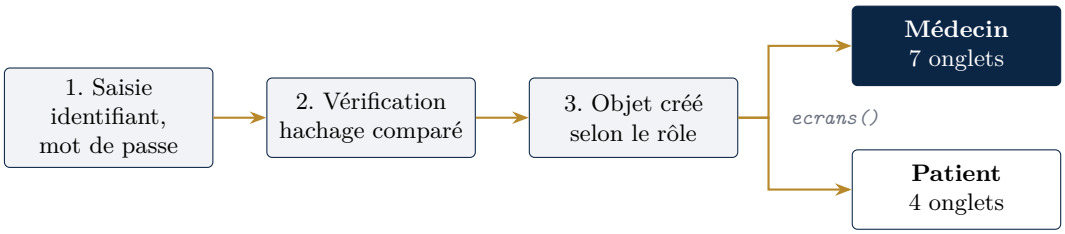


Fig. *1.* *Du mot de passe aux onglets : un seul test de rôle à la connexion.*

IV. Outils et démarche de réalisation

4.1 Technologies utilisées

Le projet utilise **exclusivement la bibliothèque standard de Python 3** (fichier `requirements.txt` : aucun paquet tiers) :

`Module`	**Usage dans le projet**
`sqlite3`	Base de données locale, requêtes paramétrées, transactions.
`tkinter, ttk`	Fenêtres, onglets, tableaux, formulaires et dessin sur `Canvas`.
`hashlib, hmac, secrets`	Hachage PBKDF2-SHA256, comparaison sécurisée, sel aléatoire.
`json`	Stockage des lignes d'ordonnance dans la colonne `ordonnance`.
`csv, webbrowser, tempfile`	Export CSV et pages HTML imprimables.
`shutil, datetime`	Sauvegarde datée de la base ; dates et heures.

Tab. *3.* *Modules de la bibliothèque standard utilisés.*

Pour la conception, nous avons utilisé **dbdiagram.io** pour dessiner le diagramme de la base de données.

4.2 Démarche de réalisation

Le travail a été mené en six étapes successives :

1. **Analyse** : lister les objets réels de la clinique et les besoins des deux profils.
2. **Base de données** : regrouper les objets en 3 tables, définir clés et contraintes.
3. **Modèle objet** : écrire les classes (`Personne`, `Patient`, `Medecin`…) et la sécurité.
4. **Accès aux données** : créer les classes DAO qui centralisent le SQL.
5. **Interface** : réaliser la connexion, les onglets et les graphiques Tkinter.
6. **Jeu d'essai et validation** : charger des données fictives, tester les scénarios, rédiger le rapport.

V. Architecture et description des fichiers

5.1 Une architecture en 3 couches

Le code est séparé en trois étages. Chaque étage ne parle qu'à celui du dessous, ce qui rend le projet facile à lire, à tester et à faire évoluer.

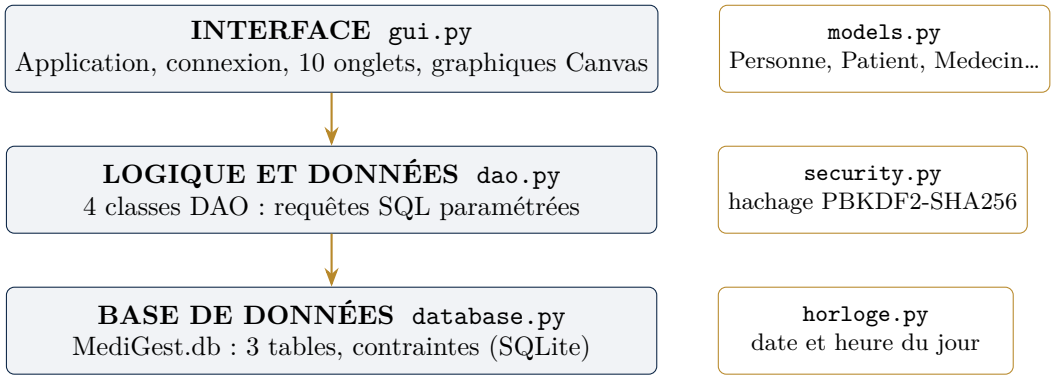


Fig. 2. *Architecture en 3 couches ; les modules de droite sont utilisés par plusieurs couches.*

5.2 Rôle de chaque fichier

Fichier	Rôle et contenu
`main.py`	**Point d'entrée.** Crée la base, ouvre la connexion, charge le jeu d'essai si la base est vide (ou recrée les comptes de test), puis lance la fenêtre `Application` et ferme proprement la base à la fin.
`gui.py`	**Interface Tkinter (environ 1 500 lignes).** Thème vert médical, composants réutilisables (`Table`, `Onglet`), graphiques sur Canvas (`Histogramme`, `Anneau`, `BarresHorizontales`), 10 classes d'onglets et la fenêtre principale `Application` (connexion, déconnexion, message d'accueil).
`dao.py`	**Accès aux données (environ 800 lignes).** Quatre classes : `UtilisateurDAO`, `RendezVousDAO`, `MedicamentDAO` et `StatistiquesDAO`. Seules ces classes écrivent du SQL.
`models.py`	**Classes du domaine médical.** `Personne`, `Patient`, `Medecin` (héritage), `RendezVous`, `Medicament`, `LigneOrdonnance`, `Ordonnance` et l'exception `RegleMetier`.
`database.py`	**Base SQLite.** Crée les 3 tables avec leurs contraintes (`creer_base`), ouvre une connexion avec accès aux colonnes par nom (`get_connection`) et teste si la base est vide (`base_vide`).
`shema.sql`	**Script SQL du schéma** utilisé pour dessiner le diagramme de la base (dbdiagram.io).
`security.py`	**Sécurité des mots de passe.** `hash_password` génère un sel aléatoire et calcule une empreinte PBKDF2-SHA256 (200 000 itérations) ; `verifier_mot_de_passe` la compare de façon sécurisée (`hmac.compare_digest`).

Fichier	Rôle et contenu
`horloge.py`	**Source unique de la date et de l'heure** (`aujourdhui`, `maintenant`, `convertir_date`).
`exports.py`	**Exports.** `exporter_csv` (CSV avec point-virgule, lisible dans Excel) et `ouvrir_dans_navigateur` (page HTML imprimable pour factures et ordonnances).
`data_test.py`	**Jeu d'essai fictif** : 2 médecins, 6 patients, 10 médicaments et 22 rendez-vous chargés automatiquement à la première exécution.
`requirements.txt`	Indique qu'aucun paquet externe n'est nécessaire.
`dbdiagram.png`	Diagramme de la base de données (3 tables, 2 relations).
`test_dashboard_*.png`	Trois captures de test du tableau de bord (`new`, `balanced`, `capped`) qui ont servi à régler les proportions des graphiques.

Tab. *4. Description de chaque fichier du projet.*

VI. Le principe d'héritage

> Un patient et un médecin sont deux sortes de **personnes** : ils ont le même début de fiche (nom, prénom, CIN, login, téléphone…). On écrit donc ce début **une seule fois** dans une classe mère `Personne`, puis `Patient` et `Medecin` y ajoutent chacun ce qui leur est propre.

6.1 Les classes du modèle

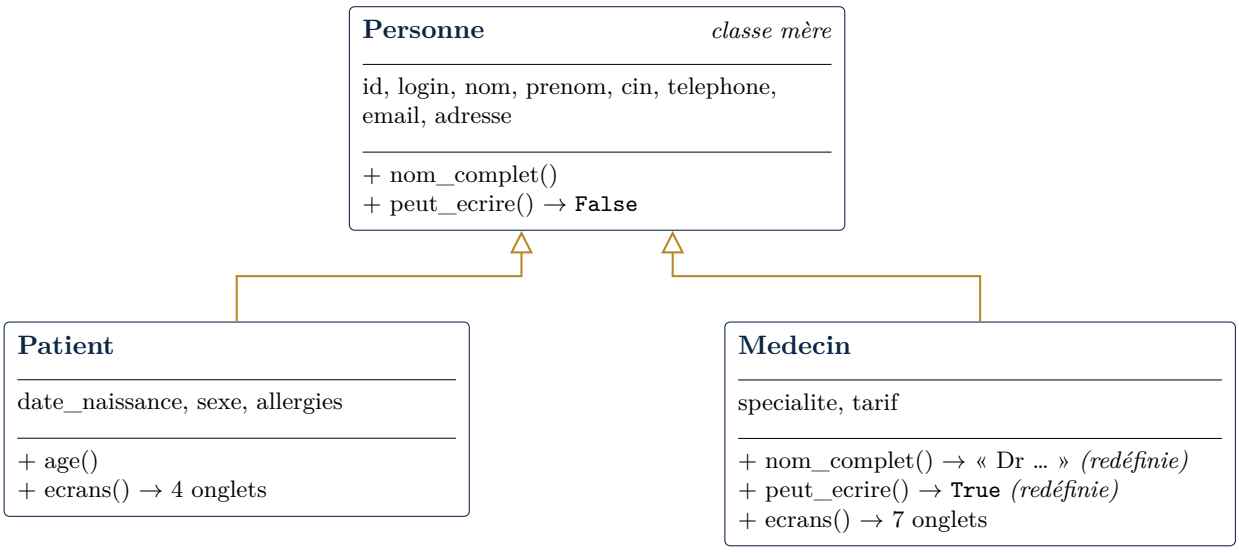


Fig. *3. Héritage : `Patient` et `Medecin` héritent de `Personne` (le triangle signifie « hérite de »).*

6.2 Le code correspondant (extrait de `models.py`)

```
class Personne:                          # classe mere
    def __init__(self, id, login, nom, prenom, cin, telephone, email, adresse):
        self.id = id
```

```
        self.login = login
        self.nom = nom
        ...
    def nom_complet(self):
        return f"{self.prenom} {self.nom.upper()}"
    def peut_ecrire(self):            # par defaut : pas de droit d'ecriture
        return False

class Patient(Personne):              # Patient herite de Personne
    def __init__(self, ..., date_naissance, sexe, allergies):
        super().__init__(id, login, nom, prenom, cin, telephone, email, adresse)
        self.date_naissance = date_naissance
        self.sexe = sexe
        self.allergies = allergies
    def ecrans(self):                 # 4 ecrans
        return ["Mes rendez-vous", "Mes ordonnances", "Mes factures", "Mon profil"
            ]

class Medecin(Personne):              # Medecin herite aussi de Personne
    def nom_complet(self):            # redefinition : "Dr ..."
        return f"Dr {self.prenom} {self.nom.upper()}"
    def peut_ecrire(self):            # redefinition : le medecin peut ecrire
        return True
    def ecrans(self):                 # 7 ecrans
        return ["Patients", "Rendez-vous", "Consultations", "Stock",
                "Facturation", "Tableau de bord", "Mon profil"]
```

6.3 Ce que l'on obtient grâce à l'héritage

Méthode	`Patient`	`Medecin`
`nom_complet()`	héritée : « Sara TAZI »	**redéfinie** : « Dr Karim ALAOUI »
`peut_ecrire()`	héritée : `False` (lecture seule médicale)	**redéfinie** : `True` (prescrire, facturer)
`ecrans()`	4 onglets (Mes rendez-vous, ordonnances, factures, profil)	7 onglets (Patients, Rendez-vous, Consultations, Stock, Facturation, Tableau de bord, Profil)
`age()`	propre au patient	absente

Tab. 5. *Héritage et redéfinition des méthodes (polymorphisme simple).*

6.4 Pourquoi c'est utile

- **Pas de répétition** : nom, CIN, login, téléphone… sont écrits une seule fois dans `Personne`.
- **Une interface qui s'adapte seule** : à la connexion, la fenêtre ne demande pas « qui es-tu ? » mais « quels écrans peux-tu ouvrir ? » (`user.ecrans()`) ; elle construit alors 7 ou 4 onglets.
- **Droits clairs** : `peut_ecrire()` sépare ce que peut faire un médecin de ce que peut faire un patient.
- **Évolution facile** : pour ajouter un nouveau profil (par exemple une secrétaire), il suffit de créer une nouvelle classe fille et de lui donner ses écrans.

6.5 Les autres notions de POO utilisées

- **Composition** : une `Ordonnance` contient une liste de `LigneOrdonnance` (médicament, quantité, prix, posologie) ; elle vérifie le stock avant d'ajouter une ligne.

— **Exception personnalisée** : `RegleMetier` hérite de `Exception` ; elle est levée quand une règle est violée (par exemple stock insuffisant).

— **Encapsulation** : chaque thème a sa classe d'accès aux données (DAO) ; l'interface appelle des méthodes sans manipuler le SQL.

— **Héritage dans l'interface** : les 10 écrans héritent d'une classe `Onglet` (méthodes `construire()` et `rafraichir()`), et `Application` hérite de `tk.Tk`.

— **Test intégré** : les fichiers `models.py` et `security.py` contiennent un petit test exécutable directement (`if __name__ == "__main__"`).

VII. Conception de la base de données

> Imaginons trois classeurs : un classeur des **personnes** (médecins et patients), un classeur des **rendez-vous** (chaque feuille raconte un rendez-vous, de la prise de date jusqu'au paiement) et un classeur des **médicaments** avec leur stock. Ces trois classeurs sont nos trois tables.

7.1 Démarche en 4 étapes

1. **Lister** les 9 objets réels : patient, médecin, rendez-vous, consultation, ordonnance, ligne d'ordonnance, facture, médicament, stock.
2. **Regrouper** ceux qui vont ensemble pour tomber à 3 tables.
3. **Relier** les tables : une clé primaire identifie une ligne, une clé étrangère renvoie vers une autre ligne.
4. **Protéger** : placer les règles dans la base elle-même (contraintes) pour qu'une erreur de programmation ne corrompe pas les données.

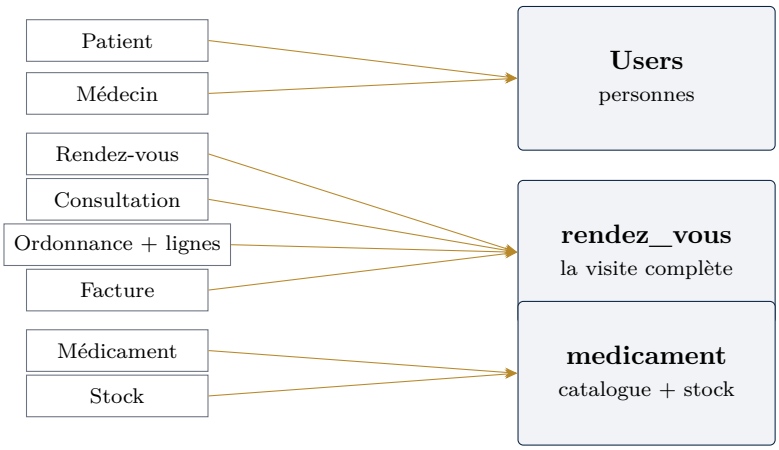


Fig. 4. *Du réel au modèle : 9 objets de la clinique regroupés en 3 tables.*

7.2 Schéma de la base

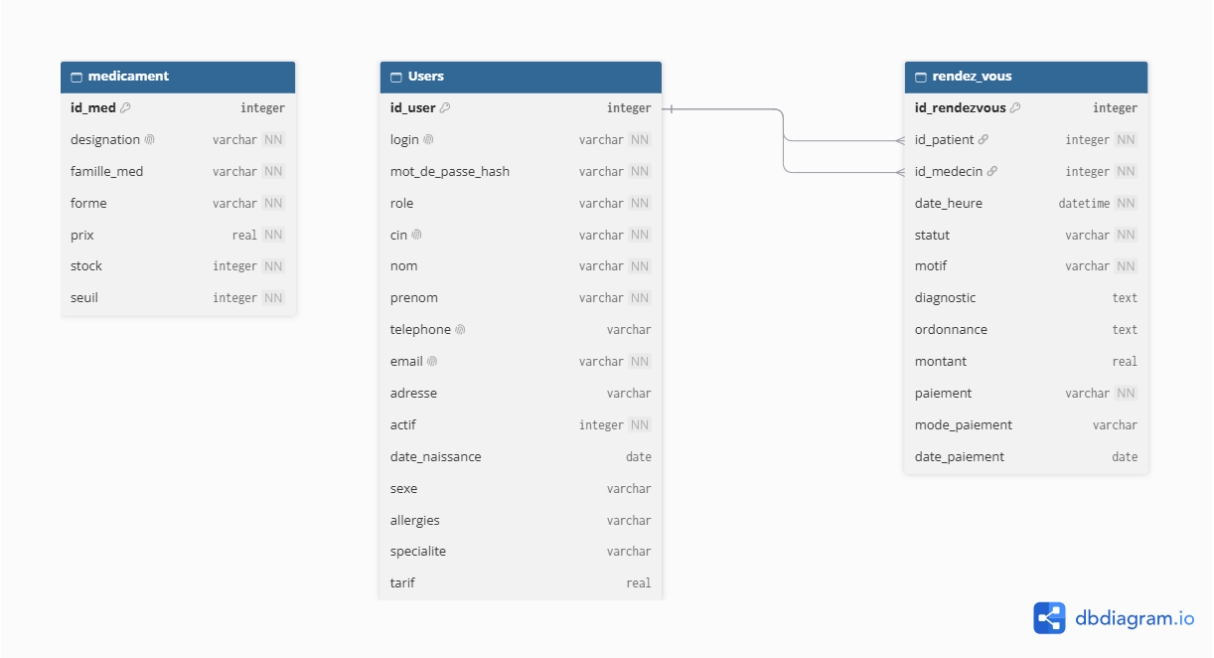


Fig. 5. *Diagramme de la base MediGest (dbdiagram.io) : `Users` est reliée deux fois à `rendez_vous` (patient et médecin) ; `medicament` est référencée par l'ordonnance.*

7.3 Description des 3 tables

Table `Users` (16 colonnes) — les personnes. Une seule table pour les deux profils : la colonne `role` (`medecin` ou `patient`) les distingue. C'est la table de la hiérarchie `Personne` → `Patient` / `Medecin`.

Table `rendez_vous` (12 colonnes) — le cœur de l'application. Une ligne correspond à un rendez-vous. Ses colonnes se remplissent au fil du temps : d'abord la date, le motif et le statut, puis après la consultation le `diagnostic` et l'`ordonnance`, puis le `montant`, et enfin le paiement. Le dossier médical d'un patient est donc la liste de ses rendez-vous honorés.

Table `medicament` (7 colonnes) — catalogue et stock. Elle est partagée par tous les rendez-vous, c'est pourquoi elle reste séparée.

Colonne	Type	Rôle et contrainte
Table Users		
id_user	INTEGER	Clé primaire, générée automatiquement.
login, cin, email	TEXT	Obligatoires et **uniques** (pas de doublon).
mot_de_passe_hash	TEXT	Empreinte du mot de passe (jamais le mot de passe).
role	TEXT	`medecin` ou `patient` (CHECK).
nom, prenom	TEXT	Obligatoires.
telephone, adresse	TEXT	Téléphone unique ; adresse facultative.
actif	INTEGER	1 = actif, 0 = archivé (archivage sans suppression).
date_naissance, sexe, allergies	TEXT	Champs du patient (`sexe` : M ou F).
specialite, tarif	TEXT, REAL	Champs du médecin ; tarif ≥ 0.
Table rendez_vous		
id_rendezvous	INTEGER	Clé primaire.
id_patient, id_medecin	INTEGER	Clés étrangères vers `Users` ; doivent être différentes.
date_heure, motif	TEXT	Date et heure (AAAA-MM-JJ HH :MM), motif de la visite.
statut	TEXT	DEMANDE, PLANIFIE, HONORE ou ANNULE (CHECK).
diagnostic	TEXT	Rempli par le médecin à la consultation.
ordonnance	TEXT	Liste des médicaments prescrits au format JSON.
montant	REAL	Tarif + médicaments ; ≥ 0.
paiement, mode_paiement, date_paiement	TEXT	`impaye` ou `paye` ; espèces, carte ou chèque ; date.
Table medicament		
id_med	INTEGER	Clé primaire.
designation	TEXT	Nom du médicament, **unique**.
famille_med, forme	TEXT	Famille (par exemple Antalgiques) et forme (comprimé, flacon…).
prix	REAL	Prix unitaire ≥ 0.
stock, seuil	INTEGER	Quantité disponible ≥ 0 et seuil d'alerte ≥ 0.

Tab. *6.* *Dictionnaire de données : colonnes des trois tables.*

7.4 Les protections intégrées à la base

Contrainte	Effet
`PRIMARY KEY AUTOINCREMENT`	Chaque ligne a un identifiant unique généré automatiquement.
`UNIQUE (login, cin, email, téléphone, désignation)`	Impossible de créer deux comptes avec le même CIN, login ou e-mail, ni deux médicaments du même nom.
`CHECK sur role, sexe, statut`	Seules les valeurs prévues sont acceptées.
`CHECK conditionnel par rôle`	Un médecin doit avoir une spécialité et un tarif ; un patient doit avoir une date de naissance et aucun tarif.
`CHECK (stock >= 0, prix >= 0, montant >= 0)`	Aucun stock ni montant négatif.
`CHECK (id_patient <> id_medecin)`	Un rendez-vous relie deux personnes différentes.
`FOREIGN KEY ON DELETE RESTRICT`	On ne peut pas supprimer une personne qui a des rendez-vous : l'historique est protégé.

Tab. 7. *Les protections de la base (fichier `database.py`).*

7.5 Le cycle de vie d'un rendez-vous

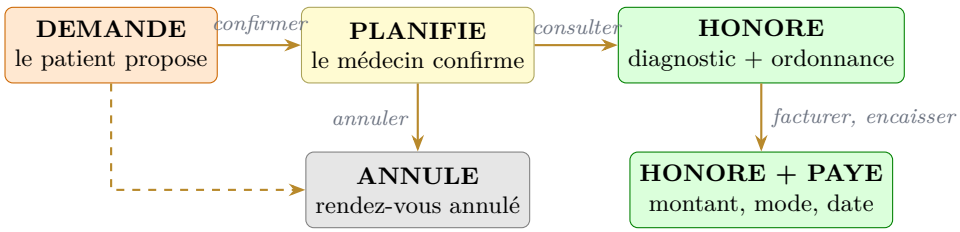


Fig. 6. *Les états d'un rendez-vous et les colonnes qui se remplissent à chaque étape.*

7.6 Interprétation des choix de conception

- **Pourquoi seulement 3 tables ?** Un rendez-vous donne au plus une consultation, qui donne au plus une facture : une seule ligne suffit pour raconter toute la visite.
- **Pourquoi une seule table `Users` ?** Un utilisateur est soit médecin, soit patient ; la colonne `role` remplace deux tables et reflète l'héritage du code.
- **Pourquoi l'ordonnance en JSON ?** Elle est enregistrée dans la colonne `ordonnance` sous forme de liste, par exemple `[{"id_med": 8, "nom": "Paracétamol 500 mg", "qte": 2, "posologie": "1 prise, 3 fois par jour", "prix": 12}]`. Le prix est figé au moment de la prescription : l'historique reste exact même si le catalogue change.
- **Pourquoi ne pas supprimer un patient ?** Son historique médical et financier doit rester consultable : on l'archive (`actif = 0`). La base refuse même la suppression d'un utilisateur lié à des rendez-vous.
- **Compromis assumé** : `rendez_vous` contient des colonnes vides avant la consultation ; les contraintes de la base compensent ce choix.

VIII. Les classes d'accès aux données (DAO)

Le principe est simple : **seules les classes DAO écrivent du SQL**. L'interface leur demande une action (par exemple « ajouter un rendez-vous ») et reçoit le résultat. Toutes les requêtes sont **paramétrées** (`?`), ce qui bloque l'injection SQL.

Classe	Principales méthodes
`UtilisateurDAO`	`chercher_par_login`, `chercher_par_id`, `ajouter_patient`, `ajouter_medecin`, `authentifier`, `afficher_patients`, `afficher_medecins`, `modifier_utilisateur`, `desactiver_utilisateur`, `reactiver_utilisateur`, `supprimer_utilisateur`, `changer_mot_de_passe`.
`RendezVousDAO`	`ajouter_rdv`, `chercher_par_id`, `afficher_tous`, `afficher_par_patient`, `afficher_par_medecin`, `modifier_statut`, `annuler_rdv`, `enregistrer_consultation`, `enregistrer_paiement`.
`MedicamentDAO`	`ajouter_medicament`, `afficher_medicaments`, `chercher_par_id`, `modifier_medicament`, `supprimer_medicament`, `modifier_stock`, `verifier_stock`, `medicaments_en_alerte`.
`StatistiquesDAO`	`nombre_patients`, `nombre_medecins`, `nombre_rdv`, `rdv_aujourdhui`, `chiffre_affaires`, `stock_en_alerte`, `resume_accueil`.

Tab. 8. *Les quatre classes d'accès aux données (`dao.py`).*

Exemple : l'authentification. La méthode `authentifier` cherche le compte par son login, refuse un compte archivé (`actif = 0`), puis compare le mot de passe saisi à l'empreinte enregistrée grâce à `security.py`. Elle renvoie la ligne de l'utilisateur ou `None` : l'interface affiche alors « Identifiant ou mot de passe incorrect ».

Exemple : la consultation. Quand le médecin valide une consultation, l'interface calcule le montant (tarif du médecin + prix des médicaments), diminue le stock de chaque médicament prescrit, puis la méthode `enregistrer_consultation` enregistre le diagnostic, l'ordonnance JSON et le montant, et passe le statut à `HONORE`.

IX. Interfaces graphiques

L'interface a été réalisée avec Tkinter et son thème `ttk` personnalisé (vert médical). À la connexion, le rôle du compte détermine la fenêtre ouverte : 7 onglets pour le médecin, 4 pour le patient. Les couleurs portent du sens : jaune = planifié, orange = demande, vert = honoré, gris = annulé, rouge = alerte.

9.1 Composants de l'interface

Classe	Rôle
`Application`	Fenêtre principale (hérite de `tk.Tk`) : connexion, déconnexion, ouverture des onglets, message d'accueil.
`Table`	Tableau réutilisable (colonnes, tri, couleurs par statut, sélection).
`Onglet`	Classe mère des 10 écrans (`construire()`, `rafraichir()`).
`OngletPatients`	Recherche, fiche patient, création de compte, archivage.
`OngletRendezVous`	Planification, filtres de l'agenda, confirmation et annulation.
`OngletConsultations`	Diagnostic, lignes d'ordonnance, rappel d'allergie, validation.
`OngletStock`	Liste du stock, alertes, réapprovisionnement, nouveau médicament.
`OngletFacturation`	Tarif, factures, encaissement, impression, export CSV.
`OngletTableauDeBord`	Six indicateurs et trois graphiques ; sauvegarde de la base.
`OngletMesRendezVous`, `OngletMesOrdonnances`, `OngletMesFactures`	Écrans du patient : ses rendez-vous (avec demande de créneau), ses ordonnances et ses factures.
`OngletProfil`	Coordonnées modifiables et changement de mot de passe.
`Histogramme`, `Anneau`, `BarresHorizontales`	Graphiques dessinés sur `Canvas` (aucune bibliothèque externe).

Tab. *9.* *Principales classes de* ***`gui.py`****.*

9.2 Connexion et espace médecin



Fig. *7.* *Connexion : identifiant et mot de passe vérifiés contre le hachage stocké.*



Fig. *8.* *Patients (médecin) : recherche, fiche avec allergies, création de compte, archivage.*

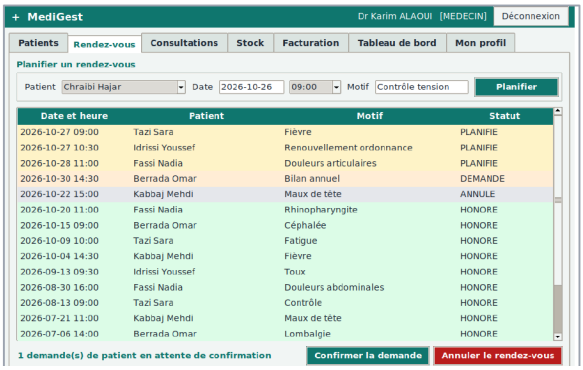


Fig. *9.* *Rendez-vous (médecin) : planification et couleurs par statut.*

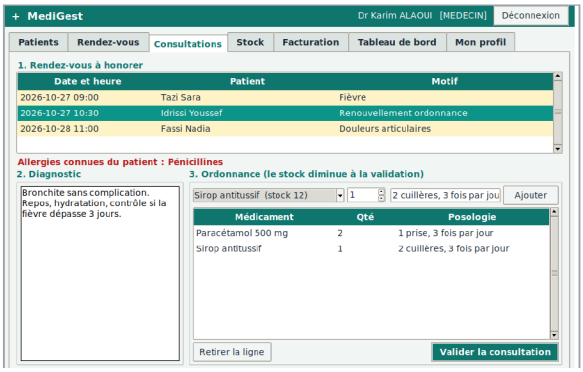


Fig. *10.* *Consultations : diagnostic, ordonnance ligne par ligne, rappel d'allergie.*

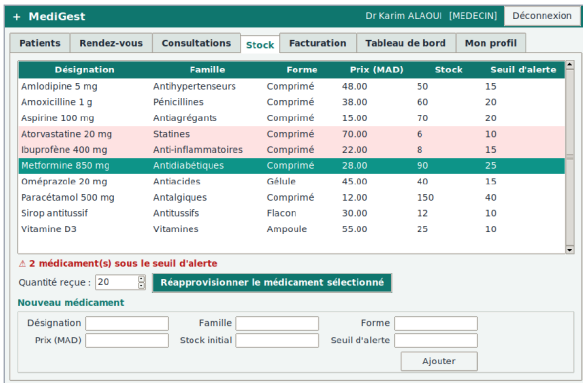


Fig. *11.* *Stock : lignes rouges sous le seuil d'alerte, réapprovisionnement.*



Fig. *12.* *Facturation : montant, encaissement, impressions et export CSV.*

9.3 Espace patient



Fig. *13.* *Mes rendez-vous : le patient demande un créneau libre.*

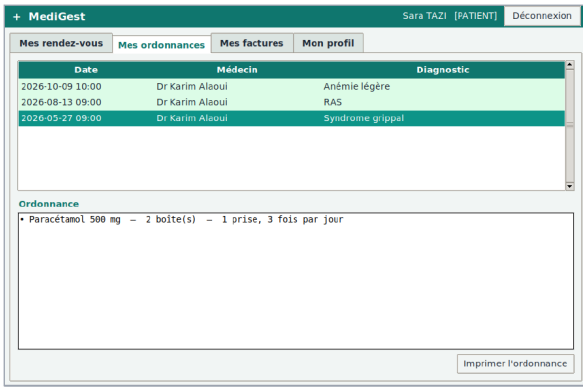


Fig. *14.* *Mes ordonnances : l'ordonnance est lue depuis le JSON.*

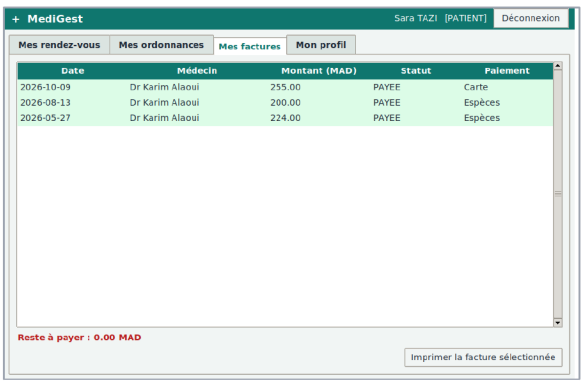


Fig. *15.* *Mes factures : montant, statut et mode de paiement.*

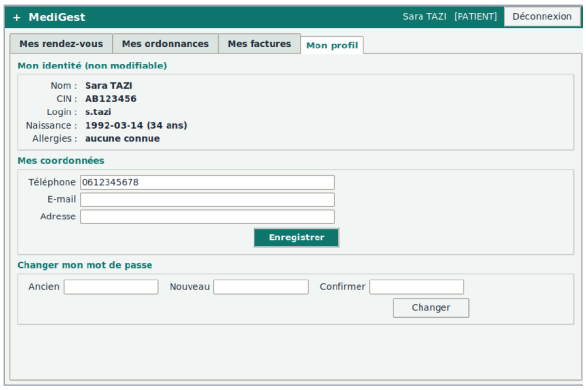


Fig. *16.* *Mon profil : identité verrouillée, coordonnées et mot de passe modifiables.*

9.4 Le tableau de bord

Le tableau de bord affiche six indicateurs (patients, consultations, chiffre d'affaires encaissé, impayés, alertes de stock, demandes en attente) et trois graphiques dessinés à la main sur `Canvas` : un histogramme du chiffre d'affaires par mois, un anneau de répartition des visites par statut et des barres horizontales pour les médicaments les plus prescrits. Les trois captures de test ci-dessous montrent comment les proportions ont été réglées pour que les graphiques s'adaptent à la fenêtre.

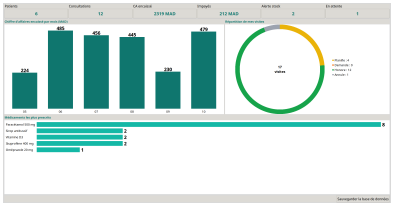


Fig. *17.* *Première version : anneau trop grand.*

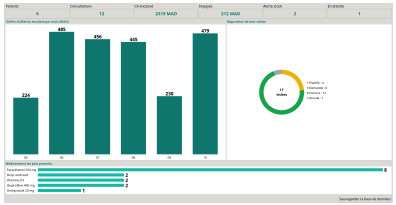


Fig. *18.* *Version plafonnée : proportions limitées.*

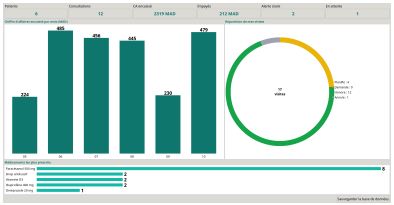


Fig. *19.* *Version retenue : équilibrée.*

X. Sécurité et protection des données

10.1 Le hachage des mots de passe

Les mots de passe ne sont **jamais** stockés en clair. À la création d'un compte, `security.py` génère un **sel** aléatoire de 16 octets puis calcule une empreinte PBKDF2-SHA256 avec 200 000 itérations ; la base ne contient que « sel$empreinte ». Deux utilisateurs ayant le même mot de passe auront donc deux empreintes différentes.

```
def hash_password(mot_de_passe):
    sel = secrets.token_hex(16)                          # sel aleatoire
    empreinte = hashlib.pbkdf2_hmac("sha256", mot_de_passe.encode("utf-8"),
                                    sel.encode("utf-8"), 200000).hex()
    return sel + "$" + empreinte                         # on stocke sel$empreinte

def verifier_mot_de_passe(mot_de_passe, stockage):
    sel, empreinte_stockee = stockage.split("$", 1)
    empreinte = hashlib.pbkdf2_hmac("sha256", mot_de_passe.encode("utf-8"),
                                    sel.encode("utf-8"), 200000).hex()
    return hmac.compare_digest(empreinte, empreinte_stockee)  # anti-timing
```

10.2 Risques et protections

Risque	Protection mise en place
Mots de passe volés	Hachage PBKDF2-SHA256, sel aléatoire, 200 000 itérations ; aucun mot de passe en clair.
Injection SQL	Requêtes paramétrées (`?`) dans toutes les classes DAO.
Accès aux données d'autrui	Le patient ne voit que ses propres rendez-vous ; seul le médecin a le droit d'écriture médicale (`peut_ecrire()`).
Perte d'historique	Archivage logique (colonne `actif`) et clés étrangères `ON DELETE RESTRICT`.
Incohérence de stock	Contrainte `stock >= 0` et vérification de la quantité avant d'ajouter une ligne d'ordonnance.
Erreur médicale	Rappel en rouge des allergies du patient pendant la consultation.
Perte de la base	Bouton « Sauvegarder la base de données » : copie datée du fichier.
Demandes abusives	Au plus 2 demandes en attente par patient ; annulation patient impossible à moins de 24 h.

Tab. *10.* *Principaux risques et protections.*

XI. Jeu d'essai et scénarios de validation

11.1 Le jeu d'essai

Le fichier `data_test.py` remplit automatiquement une base vide avec des données fictives : 2 médecins, 6 patients, 10 médicaments et 22 rendez-vous répartis de mai à novembre 2026. Il permet de tester l'application sans rien saisir.

Login	**Personne**	**Profil**	**Allergie / tarif**
dr.alaoui	Dr Karim Alaoui (médecine générale)	médecin	200 MAD
dr.bennani	Dr Salma Bennani (cardiologie)	médecin	300 MAD
s.tazi	Sara Tazi (née en 1992)	patient	aucune
n.fassi	Nadia Fassi (née en 2001)	patient	aucune
o.berrada	Omar Berrada (né en 1978)	patient	aspirine
y.idrissi	Youssef Idrissi (né en 1985)	patient	pénicillines
m.kabbaj	Mehdi Kabbaj (né en 1996)	patient	aucune
h.chraibi	Hajar Chraibi (née en 1969)	patient	aucune

Tab. *11.* *Comptes du jeu d'essai (noms fictifs).*

11.2 Scénarios de validation

Voici les principaux comportements vérifiés dans l'application, avec le résultat produit par le code :

Scénario	Résultat observé
Connexion avec un mauvais mot de passe ou un compte archivé	Accès refusé : « Identifiant ou mot de passe incorrect ».
Créer un patient avec un CIN, un login ou un e-mail déjà utilisé	Refus par la contrainte `UNIQUE` de la base ; message d'erreur affiché.
Choisir un jour et un médecin pour un rendez-vous	Seuls les créneaux de 30 min encore libres sont proposés.
Patient : 3e demande de rendez-vous en attente	Refus : « Vous avez déjà 2 demandes de rendez-vous en attente ».
Patient : annuler un rendez-vous à moins de 24 h	Refus : « Annulation impossible à moins de 24 heures ».
Valider une consultation sans diagnostic	Refus : « Veuillez saisir un diagnostic ».
Prescrire une quantité supérieure au stock	Refus : « Stock insuffisant » avec la quantité restante.
Valider une consultation avec ordonnance	Statut `HONORE`, stock diminué, montant = tarif + médicaments.
Archiver un patient ayant des rendez-vous à venir	Refus : « Impossible d'archiver un patient avec des rendez-vous à venir ».
Supprimer un utilisateur lié à des rendez-vous	Refus par la clé étrangère `RESTRICT`.

Tab. *12.* *Scénarios de validation et comportement de l'application.*

XII. Résultats et interprétation

Le tableau de bord du Dr Alaoui, calculé sur le jeu d'essai, résume l'activité de la clinique.

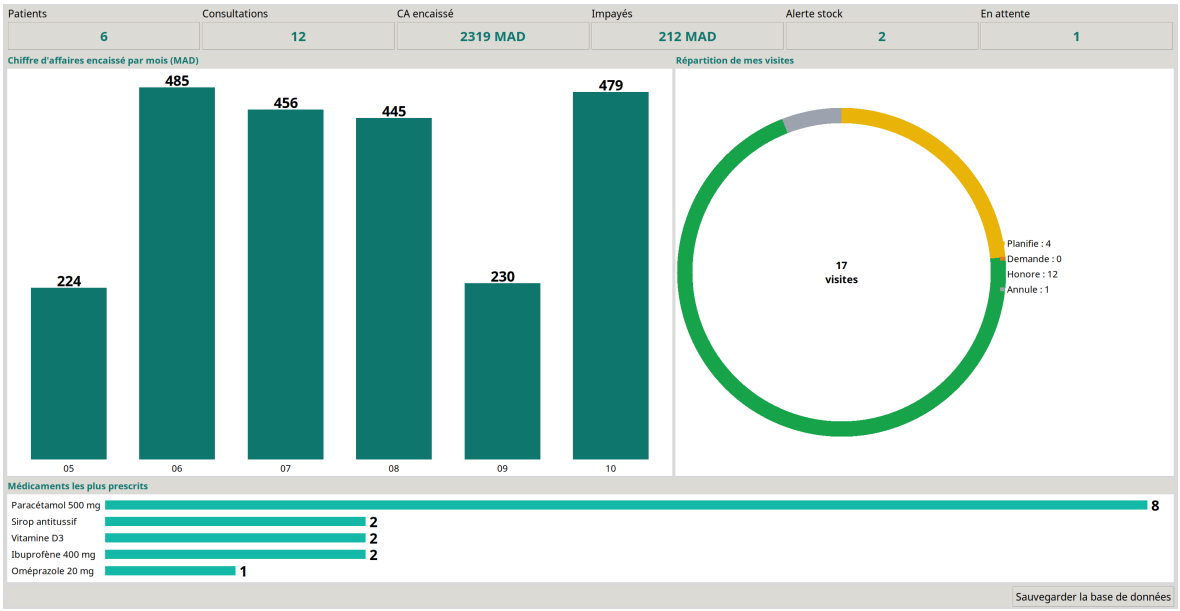


Fig. *20.* *Tableau de bord du médecin : indicateurs, chiffre d'affaires par mois, répartition des visites et médicaments les plus prescrits.*

12.1 Lecture des indicateurs

Domaine	Chiffres observés	Interprétation
Activité	17 visites : 12 honorées (71 %), 1 annulée, 4 à venir ; 1 demande en attente	Un seul rendez-vous annulé sur 17 : l'agenda est stable ; la demande en attente est signalée pour ne pas être oubliée.
Finances	2 319 MAD encaissés ; 212 MAD d'impayés	Les impayés représentent environ 8 % du total facturé (212 sur 2 531 MAD).
Chiffre d'affaires	de 224 (mai) à 485 (juin) ; 479 en octobre	Activité régulière entre 445 et 485 MAD, sauf mai (premier mois) et septembre (230 MAD), plus faibles.
Facturation	1 consultation honorée non facturée (20 octobre)	Sur le cahier papier, elle aurait probablement été perdue ; ici elle apparaît « à facturer ».
Stock	2 médicaments sur 10 (20 %) sous le seuil	Ibuprofène (6 pour un seuil de 15) et atorvastatine (8 pour 15) : l'alerte permet de réapprovisionner avant la rupture.
Prescriptions	Paracétamol : 8 unités ; les autres : 2 au plus	Le paracétamol est le médicament à surveiller en priorité.

Tab. ***13.*** *Lecture des indicateurs du tableau de bord.*

12.2 Chiffre d'affaires et stock

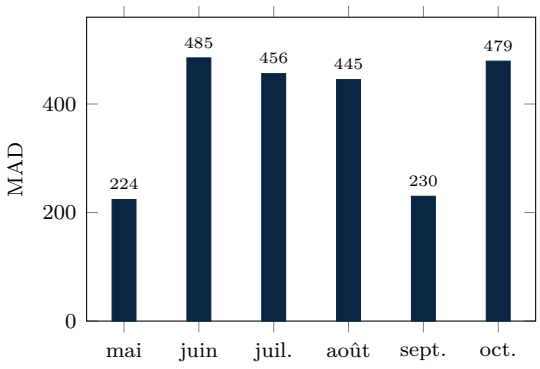


Fig. ***21.*** *Chiffre d'affaires encaissé par mois (total 2 319 MAD).*

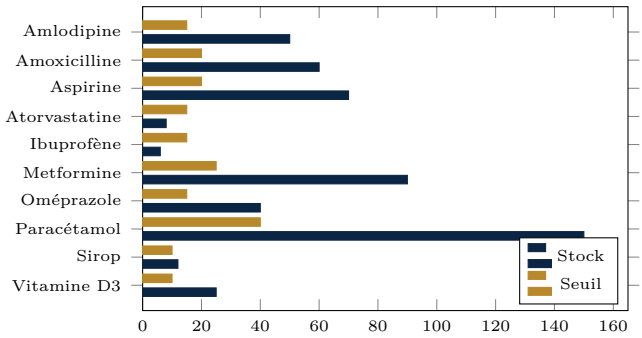


Fig. ***22.*** *Stock et seuil d'alerte par médicament.*

Le graphique de droite montre clairement les deux médicaments dont le stock (barre bleue) est inférieur au seuil (barre dorée) : l'atorvastatine et l'ibuprofène. Le sirop antitussif (12 pour un seuil de 10) est proche de son seuil et mérite aussi une surveillance.

12.3 Difficultés rencontrées et solutions

— **Respecter 3 tables sans perdre de fonctions** : consultation, ordonnance et paiement sont rattachés au rendez-vous ; les contraintes de la base compensent la largeur de la table.

— **Garder une POO claire** : une seule classe mère, quelques méthodes redéfinies et une exception dédiée, sans notions trop avancées.

— **Dessiner des graphiques sans bibliothèque externe** : histogramme, anneau et barres sont dessinés sur `Canvas`, puis leurs proportions ont été réglées grâce à plusieurs captures de test.

— **Rester dans la bibliothèque standard** : aucun paquet à installer, ce qui facilite la remise et l'exécution du projet.

— **Modèle cohérent entre code et base** : les rôles du code (`Patient`, `Medecin`) correspondent à la colonne `role` de la table `Users`.

12.4 Limites et perspectives

L'application est **monoposte** (SQLite) et la table `rendez_vous`, volontairement large, rend les évolutions de structure plus lourdes. Perspectives : tests automatiques complets des règles de gestion, export PDF des factures, rappels de rendez-vous par e-mail, sauvegarde planifiée, journal d'audit, migration vers PostgreSQL et version web.

Annexe : lancer l'application

Il suffit de Python 3 installé (avec Tkinter) ; aucun paquet à ajouter. Depuis le dossier du projet :

```
python main.py        # la base MediGest.db est creee au premier lancement
```

Comptes d'essai : médecin `dr.alaoui` / `medecin123`, patient `s.tazi` / `patient123`.

Conclusion générale

MediGest répond à l'objectif fixé : remplacer le cahier papier d'une clinique par une application simple, fiable et complète. Elle permet au médecin de planifier, consulter, prescrire, facturer et suivre son activité, et au patient de consulter son propre dossier.

Techniquement, le projet montre qu'un modèle très réduit (**3 tables**) reste solide dès lors que la rigueur est placée à la fois dans la **base de données** (contraintes, clés, valeurs uniques) et dans des **classes bien séparées**. L'**héritage** `Personne` → `Patient` / `Medecin` évite de dupliquer le code, distingue clairement les droits de chaque profil et permet à l'interface de s'adapter automatiquement. L'architecture en trois couches (interface, accès aux données, base) rend le code lisible et évolutif.

Au-delà du résultat, ce projet nous a permis de relier concrètement les notions du module « Python avancé » : programmation orientée objet, SQL, sécurité des mots de passe, interface graphique Tkinter, gestion des exceptions et travail en équipe. Il constitue une étape importante dans notre formation d'ingénieurs. Nous remercions encore Monsieur le Professeur Mustapha HAIN pour son encadrement.

ELGOTAI Abdessamad • **ELBERHICHI** Abderrahmane • **AKRINE** Youssef